Code Maintainability Benchmarking

Nicholas Guttenberg (w/ Opus 4.6 assistance)

March 2, 2026

1 Introduction

The use of LLM-based coding and coding assistance is expanding rapidly. While there are benchmarks for code accuracy [1, 2] and LLM-based judges of code quality, there is a lack of grounded evaluations of, in practice, how maintainable LLM-generated code is in the long run. If repeated application of LLM coders to a code base has some gradual but long-term deleterious effect - especially with regards to the ability of LLMs to continue to maintain that codebase - then organizations may end up in a precarious situation. So it behooves us to determine if we can quantify the maintainability of LLM-generated code and moreso ground it in specific predictions such as how much the cost to continue working on a codebase will increase over time, how hard will it become to change a certain decision as the codebase grows, and so on? This way, even if we are not benchmarking on programs comprising tens of thousands of lines of code, we might at least be able to extrapolate what will happen as codebases grow over their lifespan.

Furthermore, there are many different ways to use an LLM for coding assistance: a human engineer using it as a sounding board to write out their plans but not actually using it for coding, having an LLM write only boilerplate/individual methods, turn taking pair-coding where the human engineer copy-pastes the results into a codebase, git-based workflows with issues and pull requests with LLM agents acting autonomously to implement features but a human engineer acting as a manager, fully automated multi-agent workflows with LLMs taking on different roles and managing and validating each-other, and more. Even within those methods, there are differences in philosophy as to things such as the role of unit tests and test suites (should the human engineer write them, can an LLM write them?), code review, patterns such as plan-than-implement versus one-shot implementation, how debugging should happen (automatic via tool use, human-assisted, etc), and so on. Are LLM-written code summaries helpful - a recent study [4] shows that, despite this pattern being popular, it turns out it may increase the cost by 20% and furthermore decrease code accuracy on top of that!

Even if we would not intend to use a benchmark to drive the development of improved models, it would still be useful as a way to quantify these different approaches. And, in particular, it seems necessary not just to quantify performance on one-off coding tasks, but to measure how usage of these approaches impacts future costs and technical debt.

At this stage, we put forward an example methodology for constructing benchmarks that can evaluate multi-step development processes, specifically

looking at the differential consequences of using a methodology in one stage on the costs and issues associated with subsequent stages. We lay out a number of patterns that can occur in practice, and how one might construct benchmark tasks interrogating each of those patterns. We also discuss different metrics of interest above and beyond the usual accuracy considerations. We view factors such as human time cost, token cost, technical debt, code isolation (the degree to which development can be parallelized without generating conflicting merges), and modularity (the degree to which a developer only needs to be aware of a small part of the code to extend it) as part of a Pareto front of concerns. By identifying where each approach lies on this front, different organizations might choose different optimal procedures to suit their particular needs.

While this work does not in itself construct a full benchmark suite across all of software development as one might hope, we hope to define precisely enough how to construct tasks which interrogate these aspects so that other people may collaboratively add to the our initial benchmarks.

2 Related Work

Single-pass coding benchmarks. The dominant paradigm for evaluating LLM coding ability is single-pass code generation: given a specification or set of tests, produce correct code. HumanEval [1] and MBPP established the pass@k metric for function-level correctness, while SWE-bench [2] extended evaluation to repository-level issue resolution. LiveCodeBench addresses contamination concerns by drawing from live coding contests. These benchmarks evaluate *models*, not *methodologies*: the protocol is fixed (typically one-shot or single-agent with tool use), and the variable is which model is used. This work is complementary: here the model is fixed but the protocol is varied.

TDD and test-driven benchmarks. Recent work has begun evaluating LLMs specifically on test-driven development tasks. WebApp1K [5] uses test cases as both prompt and verification, finding that instruction following and in-context learning are more important than general coding proficiency for TDD success. TDD-Bench Verified [6] evaluates whether LLMs can generate tests *before* code is written, given only an issue description. CoreCodeBench [7] provides a configurable repository-level benchmark that can evaluate development, bug-fixing, and TDD as separate task scenarios, showing that these are distinct capabilities. These benchmarks evaluate TDD as a *task type* (can the model do it?), while our framework evaluates TDD as a *protocol choice* (does using it produce more maintainable code downstream?).

Code quality and technical debt. Several studies measure static properties of LLM-generated code. Slater [8] introduced the concept of "Architectural Erosion" in which they provided strictly specified dependency constraints ('hexagonal architecture constraints') and looked at the degree to which different models obeyed or bypassed those constraints, thus impacting the independent testability of code modules. A large-scale study of LLM-generated Java code [9] found that all evaluated models produce static analysis issues even in functionally correct code, and that practitioners prioritize maintainability over the functional correctness that benchmarks typically measure. Borg et al. [11]

found that well-maintained code (high "CodeHealth") is more amenable to AI refactoring, suggesting that code quality affects not just human but also LLM ability to work with the codebase.

These studies measure code quality as a *static property* of generated code. Our approach differs in measuring quality as a *dynamic property*: how does the code produced by one protocol affect the cost and success of future development stages?

Downstream effects of AI-assisted development. The work most closely related to ours is Borg et al. [3], who conducted a two-phase experiment in which professional developers added a feature with or without AI assistance, and then new developers had to further evolve the code. They found no significant differences in downstream development time or code quality, suggesting that AI-assisted code is neither more nor less maintainable than human-written code. However, their study did not constrain the AI methodology, so participants used whatever tools and workflows they preferred. Our framework is designed precisely to fill this gap: by comparing specific protocols against a baseline, we can ask which *ways* of using AI assistance produce more or less maintainable code.

Repository context and protocol elements. Gloaguen et al. [4] evaluated whether repository-level context files (such as `AGENTS.md`) improve coding agent performance, finding that they tend to reduce task success rates while increasing inference cost by over 20%. Mathews and Nagappan [12] found that providing human-written unit tests alongside natural language descriptions serves as a more precise specification and increases code generation success. These studies evaluate individual protocol elements (context files, test provision) in isolation. Our framework provides the structure to evaluate such elements as part of complete protocols and to measure their downstream effects.

3 Protocol

Since we are interested in evaluating protocols and not just models, we need to define some things about what constitutes a distinct and replicable protocol. In particular, we do not want to exclude protocols which may have human involvement, which means that those evaluations do depend on the human who is carrying them out.

3.1 Protocol requirements

We propose that a protocol should define the following:

Human involvement. When and in what capacity can a human intervene? Can the human contribute code? Participate in debugging? Ask or answer clarifying questions? Create implementation plans that break down the task? Make decisions between alternative options? Decide when a particular loop should stop?

For all protocols with human involvement, carefully record the total time spent by the human tester. This includes any pre-LLM "homework" such

as breaking up a specification into sub-tasks. Count any time when the human is "on-call," e.g. actively monitoring the process, even if the human is just waiting for the LLM to finish its processing. Stop the count only when the human tester can legitimately walk away from their desk for at least 10 minutes.

Stopping criteria. What ends each stage of the process? For example: maximum number of turns, unit tests pass, a token budget is exhausted, etc.

Escape valve. What happens when the process is failing to converge? For example, after a certain token budget the protocol should regress to a previous stage, give up and record the result, etc. This must be guaranteed to terminate.

Added instructions. Are there any instructions added to all specifications given to the LLM? For example, the LLM may be asked to write unit tests to accompany a specification, or to summarize the current state of the code between stages.

Per-task amendments. When the protocol involves decisions that must be adapted to the task—for example, architectural choices such as making the code modular via API-like interfaces—it is best to have one person write these per-task and distribute the amendments as part of the protocol definition.

Context management. Conditions in which contexts are cleared or split should be well-defined. For example, "start each sub-feature or bug in a clear context" vs. "debug in the same context as initial implementation."

Multi-agent patterns. Any multi-LLM or multi-role patterns should be defined. For example, first asking an LLM to write code, then asking a second LLM to take an adversarial stance and critique the code.

Tool use. Available tools (when non-standard) should be defined.

Test usage. Benchmark tasks include a training set and a holdout set of tests (see Section 5.4). Whether and how the training set tests are used should be defined as part of the protocol.

3.2 Example protocols

The following are illustrative protocols that span a range of human involvement, from fully autonomous to human-supervised. All share a common structure: the LLM is given a specification (or tests), a token budget acts as the escape valve, and the stopping criterion is either test passage or budget exhaustion. They differ in how the work is organized and who makes architectural decisions.

Direct implementation. No human intervention. The LLM is provided the specification and external tests, and asked to implement and iterate until all external tests pass or until 1M tokens have been spent per-stage.

Tests only. The LLM is only provided external tests (no specification), and asked to implement and iterate until all tests pass or 1M tokens have been spent per-stage.

Plan and implement. The LLM is provided the specification and is first asked to draft an implementation plan. The human tester can answer clarifying questions and make clarifying choices but cannot propose alternate strategies or perform any design or coding work. This can use a maximum of 3 turns, or end early if the LLM pushes the final draft. A clean context is used to implement each stage of the implementation plan, spending up to 1M tokens on each stage. The LLM is then asked to validate the result against the plan, with up to an additional 1M tokens for corrections.

Compartmentalize. The LLM is provided the specification and is first asked: "Design a code architecture for this specification which would divide up the task into isolated modules which speak to each-other over well defined interfaces." The LLM is then asked to implement that plan in a clean context (the rest—1M tokens, etc.—is the same).

Self-testing. The LLM is provided the specification and is first asked: "Devise a series of tests to confirm that code follows this specification, and a testing harness which would allow you to test the code." It is then asked to implement in a clean context, then asked "Identify the major reasons for any failed tests" in a clean context, and for each reason is provided a clean context to address that specific reason until the associated tests pass.

Manage and deploy. The LLM is provided the specification and is asked: "Devise an implementation plan. Within that plan, identify distinct tasks which can be done in parallel and tasks which depend on each-other. Then manage the implementation using a special commandline 'deploy' tool which you can provide instructions to as a text file; this will start a fresh context in which a separate instance performs whatever task is defined in your text file and returns whatever information you request. Iterate with your sub-contexts until the implementation is complete and works." The 'deploy' tool should be provided as described as part of the protocol definition. The total token budget is 10M tokens or natural completion, after which the code is evaluated in its current state.

Human supervision. The human tester reads the specification and breaks it down into a set of distinct features and stages, along with architectural choices and tests. This is written per-benchmark-task in advance and is distributed with the protocol definition. The human then turn-by-turn asks the LLM to perform each distinct stage, not showing the remainder of the task. The human decides when a stage has completed to their satisfaction. Each task must be completed within 2 wall clock hours.

4 Metrics

4.1 Approach

Rather than relying on heuristic proxies for code quality such as modularity scores, naming conventions, or static analysis warnings [8, 9] we aim to ground commonly-cited code virtues in directly measurable quantities. Properties like modularity, code isolation, and analyzability are not themselves metrics in our

framework; they are hypotheses about what makes code maintainable. Our approach is to define quantifiable base metrics, then use different *differential* benchmark structures to evaluate protocols which might be constructed to encourage those virtues in practice. We can therefore quantify the role of these code virtues in terms of grounded predictions such as 'what would it cost?' or 'how long would it take?'.

This perspective also clarifies the relationship to ISO/IEC 25010 [10], which treats analyzability, modularity, modifiability, testability, and reusability as primary maintainability properties. In our framework, some of these (e.g. testability) correspond to protocols to be evaluated (TDD and related approaches), while others (e.g. modifiability) correspond to outcomes we measure. The differential benchmark structures described below provide the bridge: they let us ask, for example, "does a protocol that prioritizes modularity actually reduce the cost and regression rate of future modifications?"

Our overall approach is inspired by [3], in which a feature was added to a codebase either with or without AI assistance, and then independently a second group of programmers was asked to extend the feature. The question was: how much did AI assistance in the first stage impact the difficulty of the second stage? In that study, the methods of using AI were unconstrained. In our case, we constrain the method so that we can compare specific protocols.

4.2 Base metrics

The base-level metrics fall into two broad categories: *cost metrics* encompassing the resources consumed during development, and *quality metrics* encompassing the functionality and correctness of the resultant code. Table 1 lists these. The remainder of this section develops a formal framework for applying these metrics to multi-stage development processes.

4.3 Notation

We represent a *development history* as an ordered sequence of stages, where each stage is written as a protocol-stage pair P_s: protocol P applied to stage s. A history is simply a comma-separated list read left to right:

$$T_a,\ B_b,\ T_c \tag{1}$$

reads "implement stage a with protocol T, then stage b with protocol B, then stage c with protocol T." The set of metrics evaluated after a given history is written $M(\cdot)$; for example, $M(T_a, B_b)$ denotes the metrics measured after the full two-stage history. When we need to specify *which* tests or features are being evaluated, we subscript M: $M_a(T_a, B_b)$ means "evaluate the tests associated with feature a after both stages."

This notation is compositional: histories are concatenated by appending stages. A history with no stages, which we write $\varnothing$, represents the code in its initial state (before any development), so $M(\varnothing)$ is the baseline measurement.

4.4 Differential structures

We define three kinds of differential benchmark structure, each designed to isolate a different aspect of how protocol choices affect code quality over time.

Table 1: Base metrics of interest.

Category	Metric	Description
Cost	Human time	Wall-clock time a human tester spends actively engaged (see Protocol §3 for counting rules)
	Token cost	Total input + output tokens consumed across all LLM calls in a stage
Quality	Test accuracy	Fraction of holdout tests passed by the final code
	Regression rate	Fraction of *previously-passing* tests that fail after a new development stage; isolates cross-stage contamination from per-stage difficulty
	Merge conflict rate	(for parallel metrics only) the number of overlaps between patches produced during parallel feature implementation
	Execution speed	Runtime of the produced code on a standardized benchmark input
	Memory usage	Peak memory consumption on the standardized benchmark input
	Code size	Lines of code (or tokens) in the final output, as a rough proxy for complexity

All three operate on the same set of base metrics (Section 4.2), but differ in the task structure they require.

4.4.1 Sequential differential

The sequential differential is the most straightforward structure. A task is divided into two or more stages - for example, implementing a feature and then extending or modifying it. We compare the effect of using protocol T versus a baseline protocol B in an earlier stage on the metrics of a later stage.

Given stages a and b, we evaluate three configurations:

$$M(B_a, B_b), \quad M(T_a, B_b), \quad M(T_a, T_b). \tag{2}$$

The *sequential differential* isolates the downstream effect of the stage-a protocol choice:

$$\Delta_{\text{seq}} M \;\equiv\; M(T_a, B_b) \;-\; M(B_a, B_b). \tag{3}$$

A positive Δ_{seq} for a cost metric means protocol T *increased* future costs; a negative value means it reduced them. The configuration $M(T_a, T_b)$ captures the combined effect of using T throughout.

For tasks with more than two stages, we can also ask more fine-grained questions using the metric subscript. Consider a three-stage task where stage b is supposed to preserve feature a, and stage c is supposed to modify feature a:

$$M_a(T_a, T_b) \quad \text{vs.} \quad M_a(T_a, T_b, T_c) \tag{4}$$

measures whether feature a's tests regress after stage c—this is a direct measurement of cross-feature contamination, and the regression rate metric (Section 4.2) captures exactly this.

The sequential differential directly speaks to *modifiability*: does the code produced by protocol T make subsequent changes easier or harder? It also captures aspects of *modularity* indirectly, since poorly modularized code will tend to require more context (higher token cost) and produce more regressions when modified.

4.4.2 Parallel differential

Some aspects of code quality only manifest when multiple developers (or agents) work on the codebase concurrently. The parallel differential is designed to measure *code isolation*: the degree to which independent features can be developed in parallel without generating conflicts.

We introduce one additional piece of notation: $B_b \| B_c$ denotes stages b and c implemented independently (in isolated contexts, on the same starting codebase) and then merged. A parallel differential task first establishes a codebase with stage a, then branches:

$$\Delta_{\text{par}} M \;\equiv\; M(T_a,\; B_b \| B_c) \;-\; M(B_a,\; B_b \| B_c). \tag{5}$$

The metrics of interest after the merge include post-merge test accuracy, regression rate, and merge-specific costs such as conflict rate or tokens spent resolving conflicts. This structure requires tasks specifically designed with parallelizable features (see Section 5).

The parallel differential grounds the notion of *code isolation*: if protocol T produces well-separated modules with clean interfaces, independent feature branches should merge with fewer conflicts and fewer regressions.

4.4.3 Round-trip differential

The round-trip differential applies a transformation that is nominally the identity - the code is transformed and then the transformation is reversed - and measures the deviation from the starting point. Because the expected result is known (no change), this structure does not require a baseline protocol, which eliminates a potential confound.

We can express this using an inverse. If T_a denotes implementing stage a with protocol T, then T_a^{-1} denotes the reversal: translating back to the original language, restoring the original structure after a refactoring, and so on. A round-trip is then:

$$R_a \;\equiv\; T_a,\; T_a^{-1} \tag{6}$$

and the round-trip differential is:

$$\Delta_{\text{rt}} M \;\equiv\; M(T_a, T_a^{-1}) \;-\; M(T_a). \tag{7}$$

If T_a admitted a true inverse, $M(T_a, T_a^{-1})$ would equal $M(\varnothing)$—the metrics of the original code. In practice, LLM-mediated transformations are lossy, and Δ_{rt} quantifies this loss. Note that T_a^{-1} is not literally the inverse operation but the best available approximation: translating Python $\rightarrow$ Rust $\rightarrow$ Python involves

two distinct forward translations, not an undo. This is part of why $\Delta_{\mathrm{rt}} \neq 0$ in general.

Natural choices for the round-trip transformation vary:

- **Refactor and restore**: apply a structural refactoring (extract methods, reorganize modules) and then reverse it. This tests whether the code's architecture is robust or whether it has implicit dependencies not surfaced by the structure.
- **Language translation**: translate the code to another language and back (e.g. Python → Rust → Python). This is a frequent use-case of automated LLM coding, and tests whether the code's semantics survive re-expression in different idioms.
- **Addition and removal**: A feature could be added, expanded upon, and then the base feature removed. So here we would be measuring $M(T_a, T_b, T_a^{-1}) - M(T_b)$.

More severe round trips are possible (e.g. translating between paradigms: OOP ↔ functional), but at some point the round-trip differential begins to measure the LLM's translation ability rather than the code's structural quality, so there is a practical ceiling on useful severity.

The round-trip differential speaks to *analyzability*: code that is clear and well-structured should survive round trips with less degradation, because the LLM can more reliably parse and reconstruct its intent. Whereas the sequential and parallel differential metrics measure the impact of protocols on each-other, the round-trip differential directly compares protocols against each-other - $\Delta_{\mathrm{rt}} M$ under T versus under B - to ask whether a particular protocol produces more structurally robust code independent of any inter-protocol interactions.

4.5 Grounding code virtues

Table 2 summarizes how the commonly-cited maintainability virtues map onto our measurable framework. Rather than measuring these virtues directly via heuristics, we use the differential structures to ask whether protocols that claim to promote these virtues actually produce measurably better outcomes.

5 Tasks

This section describes how to construct benchmark tasks that produce informative differential metrics. We do not attempt to define a comprehensive benchmark suite; rather, we specify what a well-formed task looks like so that tasks can be contributed incrementally. We will create and test a set of example tasks in a separate but connected work.

5.1 Task anatomy

A benchmark task consists of:

Domain. A self-contained problem domain with a specification. The domain should be complex enough that architectural choices matter - a 50-line

Table 2: Mapping code virtues to grounded measurements. Each virtue is associated with the differential structure(s) that can detect its presence and the base metrics most sensitive to it.

Virtue	Differential type	Sensitive base metrics
Modifiability	Sequential	Token cost, accuracy, and regression rate in stage 2
Modularity	Sequential	Token cost (context needed), regression rate
Code isolation	Parallel	Merge conflict rate, post-merge accuracy, post-merge regression rate
Analyzability	Round-trip	Test accuracy and code size after round trip
Testability	-	(Corresponds to protocols such as TDD [5, 6]; not a metric)

utility function is unlikely to reveal differential effects - but constrained enough that a complete implementation is feasible within reasonable token budgets. Domains with layered features (where later features build on or interact with earlier ones) are particularly useful, since they provide natural stage boundaries.

Stage sequence. An ordered sequence of stages, where each stage asks the LLM to perform a specific kind of work on the codebase (see Section 5.2). The sequence defines the pipeline that will be evaluated. A single domain may support multiple stage sequences exercising different differential structures. Stages that can be performed independently on the same starting codebase should be annotated as *parallelizable*, enabling automatic generation of both sequential and parallel pipeline variants from one task definition.

Test pool. A set of tests divided into a *training set* (visible to the LLM during development, for protocols that use tests) and a *holdout set* (used only for evaluation). See Section 5.4 for design principles.

Tags. Metadata describing what the task is designed to probe, drawn from the canonical tag set (Table 3) and extended as needed. Tags enable post-hoc analysis: for example, "protocol T outperforms B on sequential differentials, but only for tasks tagged `implicit-invalidation`."

5.2 Stage types

Each stage in a pipeline asks the LLM to perform a specific kind of work. We identify eight stage types, each characterized by what the LLM is given and what it is expected to produce. Feature addition to the null state (i.e. implementing from scratch) is simply the first feature addition in a pipeline, so we do not treat initial implementation as a separate type.

Feature addition. Given existing code (possibly empty) and a feature specification, extend the implementation. This is the most common stage type

Table 3: Canonical tag set. Task authors may add additional tags, but should use these where applicable to ensure comparability across tasks. Tags are organized by what they describe.

Category	Tag	Meaning
Domain	`backtracking`	Correct implementation requires search or backtracking
	`stateful`	Domain involves mutable state across operations
	`concurrent`	Domain involves concurrency or parallel execution
	`parser`	Domain involves parsing structured input
	`numerical`	Domain involves numerical computation or floating-point concerns
Pipeline	`implicit-invalidation`	A later stage silently invalidates assumptions from an earlier stage
	`parallel-merge`	Pipeline includes stages designed to be run in parallel and merged
	`round-trip`	Pipeline includes a round-trip transformation
	`feature-churn`	Pipeline includes both addition and removal of features
	`refactor-then-extend`	Pipeline includes a refactoring stage followed by feature addition
Stage	`debugging`	Pipeline includes a debugging stage with mutation-introduced bugs
	`integration`	Pipeline includes an integration stage with external code
	`interface-underspec`	Integration stage exercises an incomplete interface
	`interface-overspec`	Integration stage exercises a "false promise" in the interface
	`optimization`	Pipeline includes an optimization stage with performance targets
	`simplification`	Pipeline includes a simplification stage

and the default building block of sequential pipelines. Tests for feature addition are directly related to the function of the feature.

Feature removal. Given existing code and a removal specification, remove a feature and clean up. Feature removal followed by feature addition tests whether a protocol can gracefully evolve a codebase rather than only grow it. Tests for feature removal might involve definition of specific program behavior that should occur only in the absence of the feature (e.g. if a UI should have one button per feature, there should be one fewer buttons) as well as direct checks against the codebase for things which should not exist except if that feature were to be present. Verification that tests associated with that particular feature now fail is an option, but that could happen in a variety of ways, so is a weaker evaluation.

Refactoring. Given existing code and a target architecture or structural constraint, restructure the code without changing its external behavior. This is a cost that should pay for itself in reduced cost at subsequent stages. The sequential differential $\Delta_{\text{seq}}M$ directly measures whether it does: compare the cost of stage $n+1$ with and without a preceding refactoring stage. Tests here are mainly about regression - changes in the metrics such as lines of code evaluate the quality of the refactor.

Translation. Given existing code and a target language, produce an equivalent implementation in the new language. This is the basis of the round-trip differential when paired with a reverse translation. The tests here can simply be the same tests as the base program, but in the new language.

Debugging. Identify and fix bugs without introducing regressions. Rather than requiring task authors to hand-craft buggy code, we recommend using *mutation testing*: take the output of stage n, apply a mutation testing tool (e.g. mutmut for Python, pitest for Java) to introduce k controlled mutations, and present the mutated code along with failing tests as stage $n+1$. Because the mutations are applied to the protocol's own output, this directly measures how debuggable that output is: well-structured code with clear control flow should make mutations easier to locate, while tangled code should make them harder. The number and severity of mutations can be calibrated per-task. As a secondary measurement, the fraction of mutations caught by the training tests vs. the holdout tests provides a mutation coverage score, which is informative for evaluating test-driven protocols.

Integration. Given existing code with a defined interface, and a fixed piece of external code that was written against a reasonable but imperfect reading of that interface, make the two work together. The task author provides the external code, written to follow the interface's documented contract but deliberately exercising assumptions that a naive implementation might not support. The LLM may modify the existing code, the interface, the external code, or some combination - and what it chooses to modify is itself informative.

Integration stages are designed to probe the boundary between what an interface promises and what the implementation actually supports. Two characteristic kinds of interface gap produce informative tasks:

- *Interface underspecification*: the API does not expose something the external code needs. The integration task then requires either extending the interface or finding a workaround. This tests whether the protocol produced an interface that is extensible without breaking existing consumers.
- *Interface overspecification* (or "false promises"): the API implies a contract that the implementation does not fully honor - for example, an "event stream" that is actually only used for dependency resolution, or a thread-safety guarantee that does not hold for singleton resources. The integration task requires diagnosing why correctly-written client code does not work as expected.

For overspecification it may be difficult to specifically target a particular set of interface gaps here as part of the test can involve choices that the LLM makes when deciding how to architect things (in looser protocols at least), so complexity may be the best guide there. For underspecification the task spec can be written to intentionally leave interfaces lacking features which the task author knows will be likely needed for the subsequent code (testing the foresight of the human/LLM pair in a given protocol). Tests here are basically the same as with feature addition, though there can be the added feedback of seeing which tests pass automatically (e.g. without any additional integration work needing to be done).

Optimization. Given existing code and a performance target (e.g. speed, memory usage), improve the implementation to meet the target while preserving correctness. Optimization is distinct from refactoring (which changes structure without targeting performance) and from simplification (which reduces complexity). Optimized code may be *more* complex than the original, which creates a natural tension: the sequential differential after an optimization stage measures whether the performance gains come at the cost of increased maintenance difficulty. Performance targets make reasonable tests here as long as the machine used for testing is standardized, or if performance is converted to a machine-independent quantity (number of operations per loop, etc).

Simplification. Given existing code that passes all tests, reduce its complexity while preserving behavior. Concrete simplification tasks include merging duplicated or near-duplicated code, removing dead code, eliminating unnecessary abstractions, and consolidating redundant logic. The base metrics of interest are code size (which should decrease) and token cost (the cost of performing the simplification), while test accuracy and regression rate should remain unchanged.

In practice, simplification is a surprisingly common source of regressions: "dead" code turns out to have side effects, "duplicated" functions have subtle differences, and shared state creates hidden dependencies. The regression rate after a simplification stage is therefore a sharp test of whether the preceding protocol produced code whose dependencies are transparent. If the LLM cannot safely simplify the code, that is evidence that the code's structure does not make its invariants clear.

5.3 Designing for differentials

Not every sequence of stages produces an informative differential. The key design constraint is that stages must *interact*: the protocol choice in an earlier stage must be capable of affecting outcomes in a later stage. If stages are completely independent, the differential will be zero regardless of protocol, and the task is uninformative.

For **sequential differentials**, stages should share state in ways that create opportunities for both positive and negative transfer. Good sequential tasks have later stages that require understanding or modifying code from earlier stages. For example, a feature addition stage followed by a feature removal stage forces the LLM to engage with the existing implementation rather than simply appending new code.

A particularly informative sequential pattern is *implicit invalidation*: a later stage changes an assumption that earlier code depends on, without explicitly stating what needs to change. For example, if stage 1 implements a memoization cache keyed on content, and stage 2 changes the data model so that position matters, the cache is silently invalidated. The metric $M_a(T_a, T_b)$—evaluating stage a's tests after stage b—directly measures whether the protocol catches this. Tasks designed around implicit invalidation should be tagged accordingly, as this is a qualitatively different challenge from straightforward feature extension.

For **parallel differentials**, the features implemented in parallel must be *plausibly but not inherently independent*. If they are truly independent (touching completely disjoint code), every protocol will succeed at the merge; if they are inherently entangled (requiring changes to the same functions), every protocol will produce conflicts. The informative region is where good architectural choices in the initial stage make the features independent, and poor choices make them collide. This is a demanding design constraint, and task authors should verify empirically (by running a few protocols) that the parallel differential is non-trivial.

For **round-trip differentials**, any sufficiently complex codebase can be round-tripped, but the task should be complex enough that a naive translation loses something. Code with implicit structural dependencies, non-obvious invariants, or idiom-heavy implementations will produce more informative round-trip differentials than straightforward procedural code.

5.4 Test pool design

Each task includes a pool of tests divided into a training set and a holdout set. The training set is available to protocols that use tests during development (e.g. test-driven protocols, or protocols where the LLM iterates until tests pass). The holdout set is used only for evaluation and is never shown to the LLM.

The critical design principle is that the holdout tests should exercise behaviors that are not trivially implied by the training tests. A random split is insufficient: tests for a given feature tend to be correlated. Instead, we recommend a structural split:

Training tests should cover the specification's stated behavior: the "happy path" for each feature, basic input validation, and straightforward compositions of features. These are the tests a competent human developer would write from reading the spec.

Holdout tests should cover edge cases, implicit interactions between features, and behaviors that the spec does not fully enumerate. These are the tests that require understanding of the domain beyond what the spec states explicitly.

This design ensures that even a protocol which relies heavily on training set passage still faces meaningful evaluation on behaviors that require genuine understanding. The gap between training accuracy and holdout accuracy is itself an informative metric: a large gap suggests the protocol is overfitting to the visible tests rather than implementing the specification faithfully.

When this breakdown is hard to identify, it may be useful to consider mutation testing to try to at least identify classes of tests which tend to pass or fail together, and use that to separate out the training set from the holdout set.

References

[1] M. Chen, J. Tworek, H. Jun, et al., "Evaluating Large Language Models Trained on Code," *arXiv preprint arXiv:2107.03374*, 2021.

[2] C. E. Jimenez, J. Yang, A. Wettig, et al., "SWE-bench: Can Language Models Resolve Real-World GitHub Issues?" *Proc. ICLR*, 2024.

[3] M. Borg, D. Hewett, N. Hagatulah, et al., "Echoes of AI: Investigating the Downstream Effects of AI Assistants on Software Maintainability," *arXiv preprint arXiv:2507.00788*, 2025.

[4] T. Gloaguen, N. Mündler, M. Müller, V. Raychev, M. Vechev, "Evaluating AGENTS.md: Are Repository-Level Context Files Helpful for Coding Agents?" *arXiv preprint arXiv:2602.11988*, 2026.

[5] Y. Cui, "Tests as Prompt: A Test-Driven-Development Benchmark for LLM Code Generation," *arXiv preprint arXiv:2505.09027*, 2025.

[6] T. Ahmed, M. Hirzel, R. Pan, A. Shinnar, S. Sinha, "TDD-Bench Verified: Can LLMs Generate Tests for Issues Before They Get Resolved?" *arXiv preprint arXiv:2412.02883*, 2024.

[7] L. Fu, H. Guan, B. Zhang, et al., "CoreCodeBench: A Configurable Multi-Scenario Repository-Level Benchmark," *arXiv preprint arXiv:2507.05281v2*, 2026.

[8] T. Slater, "Quantitative Analysis of Technical Debt and Pattern Violation in Large Language Model Architectures," *arXiv preprint arXiv:2512.04273*, 2025.

[9] X. Sun, D. Ståhl, K. Sandahl, C. Kessler, "Quality Assurance of LLM-generated Code: Addressing Non-Functional Quality Characteristics," *arXiv preprint arXiv:2511.10271*, 2025.

[10] ISO/IEC, "Systems and software engineering—Systems and software Quality Requirements and Evaluation (SQuaRE)," *ISO/IEC 25010:2023*, 2023.

[11] M. Borg, N. Hagatulah, A. Tornhill, E. Söderberg, "Code for Machines, Not Just Humans: Quantifying AI-Friendliness with Code Health Metrics," *arXiv preprint arXiv:2601.02200*, 2026.

[12] N. S. Mathews, M. Nagappan, "Test-Driven Development and LLM-based Code Generation," *Proc. 39th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pp. 1583–1594, 2024.